Curso Avançado

Capítulo 8 - Tratamento de falhas e situações de emergência

Interromper a progressão da operação, preservar os recursos protegidos e recuperar o sistema somente após conferência.

Introdução	Uma automação precisa lidar com informações incoerentes, ausência de resposta e pedidos de emergência. A reação não termina no comando de parada: o sistema também deve manter bloqueios e orientar a recuperação.
Objetivo	Organizar detecção, registro, pedido de parada, aviso e rearme, sem retomar movimentos automaticamente.
Meta de aprendizagem	Testar uma falha persistente, reconhecer a diferença entre rearme e liberação de recursos e registrar a causa antes de reiniciar a operação.

1. A falha altera o que o sistema pode autorizar

Uma informação perdida pode impedir que o programa saiba se um trecho está livre ou se um desvio está na posição exigida. Nessas condições, não é suficiente exibir uma mensagem e continuar a sequência normal.

O programa deve bloquear decisões que dependam da informação inválida. As reservas existentes permanecem protegidas até a conferência da composição, dos recursos e da causa da falha.

No exemplo deste capítulo, qualquer falha bloqueia a operação simulada e solicita parada. Essa escolha simplifica o estudo. Um projeto real pode ter reações locais e globais, desde que sua abrangência seja definida e validada.

2. Pedido de parada não confirma parada física

Uma mensagem ou solicitação zero não comprova que o trem parou. A recuperação depende de verificar a condição física. O exemplo não aciona tração, relés, buzzer ou LEDs.

3. Exemplos de falhas e resposta planejada

Ocorrência	Resposta a definir e testar
Evento fora de ordem	Bloquear progressão, pedir parada e manter reservas.
Posição do desvio desconhecida	Impedir autorização dependente desse desvio e proteger a região.
Comunicação sem atualização	Considerar os dados dependentes inválidos. Não presumir trecho livre.
Prazo de etapa excedido	Registrar a etapa e impedir avanço automático.
Reinício ou falta de energia	Iniciar bloqueado e reconstruir a situação por conferência.
Emergência solicitada	Executar a ação de parada prevista e manter o bloqueio até recuperação.

Um Hall continuamente acionado pode representar locomotiva parada, ímã próximo ou problema de instalação. A interpretação depende da etapa e do tempo previsto para aquele ponto. O nível contínuo não deve gerar novas passagens repetidamente.

4. Emergência e proteção física

A emergência da EFMN 74 prevê aviso visual e sonoro. Esses avisos devem indicar a condição bloqueada e permitir reconhecer a ocorrência. A confirmação de leitura do aviso não elimina sua causa.

A ação elétrica de emergência deve seguir o projeto de tração e proteção. Um comando de software pode deixar de funcionar se o controlador, sua alimentação ou a comunicação falhar. Por isso, a proteção física deve ser verificada separadamente da lógica de simulação.

Não improvise conexões de emergência com base neste código. Ele ensina a sequência das decisões e não especifica circuito de corte, interface de central DCC ou atuação de relés.

5. Primeiro registro e ocorrências posteriores

Preserve a primeira causa reconhecida. Ela pode explicar os eventos que vieram depois. Registre também ocorrências adicionais, mantendo clara a diferença entre causa inicial e consequências.

6. Recuperação em etapas

Etapa	Conferência ou ação
Reconhecer a ocorrência	Consultar o motivo e a etapa. Se houver aviso sonoro, reconhecer não significa rearmar.
Estabilizar a situação	Conferir a parada e impedir novos movimentos.
Corrigir e inspecionar	Verificar a causa, a posição dos trens, os desvios e os recursos.
Rearmar	Retirar a condição de falha após inspeção, mantendo reservas e sem autorizar movimento.
Liberar recursos	Conferir a saída completa e a disponibilidade de todos os recursos protegidos.
Nova operação	Preparar e autorizar uma nova sequência, com todas as verificações.

O comando TESTE/REARME previsto no projeto deve representar uma sequência definida. Acioná-lo não pode apagar informações necessárias à recuperação nem equivaler a uma nova partida.

7. Exemplo didático de comunicação

H representa o recebimento de uma informação atualizada do controlador. Durante preparação ou circulação, mais de cinco segundos sem H gera falha. O exemplo não transmite mensagens reais nem verifica equipamentos externos.

Receber H depois da falha não rearma o sistema. O bloqueio permanece até a inspeção. Uma atualização válida deve pertencer à operação e ao controlador esperado, regra que será aprofundada no capítulo seguinte.

8. Comandos do exercício

Comando	Função
I / P / G	Inspeção inicial / reservar recursos / iniciar operação simulada.
H / A	Atualização simulada / evento de chegada e pedido de parada.
E / X	Simular evento incoerente / emergência.
R	Rearmar após correção, parada e inspeção. Não libera recursos.
L	Conferir recursos livres e liberar, após chegada ou rearme.

9. Código em C++ - estado bloqueado e registro da falha

Reúna as duas partes em um único sketch. Use o Monitor Serial em 115200 baud. Os avisos são mensagens de simulação, sem saídas físicas.

```
enum class State { STARTUP, IDLE, RESERVED, ACTIVE,
                   WAIT_CLEAR, FAULT, RECOVERY };
State state = State::STARTUP;
bool resourcesHeld = false;
const char* firstFault = nullptr;
unsigned long lastHeartbeat = 0;
unsigned long stageStarted = 0;
const unsigned long heartbeatTimeoutMs = 5000UL;
const unsigned long stageTimeoutMs = 60000UL;

void enterState(State next) {
  state = next;
  stageStarted = millis();
}
void requestStop() {
  Serial.println("SIMULATION: stop request.");
}
void raiseFault(const char* reason) {
  if (firstFault == nullptr) firstFault = reason;
  requestStop();
  enterState(State::FAULT);
  Serial.println("SIMULATION: visual and audible fault alert.");
  Serial.print("First fault: "); Serial.println(firstFault);
  Serial.print("Reported event: "); Serial.println(reason);
  // Preserve resourcesHeld. Do not resume automatically.
}
void handleCommand(char c) {
  if (c == 'X' || c == 'E') {
    raiseFault(c == 'X' ? "Emergency" : "Unexpected sensor event");
    return;
  }
  if (c == 'H') {
    lastHeartbeat = millis();
    Serial.println("Simulated fresh update. No automatic reset.");
    return;
  }
  if (c == 'I' && state == State::STARTUP) {
    // Operator confirms stopped trains and all resources clear.
    resourcesHeld = false;
    firstFault = nullptr;
    enterState(State::IDLE);
    Serial.println("Initial inspection accepted.");
  } else if (c == 'P' && state == State::IDLE) {
    resourcesHeld = true;
    lastHeartbeat = millis();
    enterState(State::RESERVED);
    Serial.println("Resources reserved. Await G.");
  } else if (c == 'G' && state == State::RESERVED) {
    // Simulation: operator has checked the complete route.
    enterState(State::ACTIVE);
    Serial.println("SIMULATION: operation started.");
```

10. Código em C++ - rearme e liberação separados

```
  } else if (c == 'A' && state == State::ACTIVE) {
    requestStop();
    enterState(State::WAIT_CLEAR);
    Serial.println("Arrival event. Reservations retained.");
  } else if (c == 'R' && state == State::FAULT) {
    // Operator confirms cause corrected and trains stopped.
    // Physical situation and all relevant data were inspected.
    Serial.print("Reviewed fault: "); Serial.println(firstFault);
    firstFault = nullptr;
    enterState(State::RECOVERY);
    Serial.println("Rearmed. No movement. Reservations retained.");
  } else if (c == 'L' &&
             (state == State::WAIT_CLEAR || state == State::RECOVERY)) {
    // Operator confirms ALL protected resources physically clear.
    resourcesHeld = false;
    enterState(State::IDLE);
    Serial.println("Resources released. Await new preparation.");
  } else if (state == State::FAULT) {
    Serial.println("Fault latched. Inspect before R.");
  } else {
    raiseFault("Unexpected command");
  }
}
void checkTimeouts() {
  unsigned long now = millis();
  if (state == State::RESERVED || state == State::ACTIVE) {
    if (now - lastHeartbeat >= heartbeatTimeoutMs) {
      raiseFault("Communication update timeout");
      return;
    }
  }
  if (state == State::RESERVED || state == State::ACTIVE ||
      state == State::WAIT_CLEAR) {
    if (now - stageStarted >= stageTimeoutMs)
      raiseFault("Operation stage timeout");
  }
}
void setup() {
  Serial.begin(115200);
  requestStop();
  Serial.println("STARTUP: inspect before I.");
}
void loop() {
  checkTimeouts();
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
}
```

A preparação inicia uma janela didática de cinco segundos para atualização. Em um sistema real, a comunicação deve estar válida antes da autorização. Não use essa janela como prova de disponibilidade do controlador.

R representa inspeção manual. O código não verifica sozinho a correção da causa. Se houver composição nos recursos, L não pode ser enviado. A ocupação deve continuar protegida até sua retirada controlada.

11. Roteiro de testes

Teste	Resultado esperado
I P H G H A L	Ciclo normal, com chegada, parada conferida e liberação.
I P G; não enviar H por 5 s	Falha por ausência de atualização. Reserva mantida.
H depois da falha	Atualização recebida. A falha continua bloqueada.
R após inspeção	RECOVERY. Sem movimento e sem liberação de recursos.
G em RECOVERY	Comando inválido. Novo bloqueio, sem partida.
L em RECOVERY, recursos livres	Liberação e retorno a IDLE. Exige nova preparação.
X durante circulação	Pedido de parada e aviso de falha. Reserva mantida.
E depois de X	Primeiro motivo permanece Emergency; novo evento também é mostrado.

Execute os comandos de comunicação dentro da janela do teste. Para verificar o prazo de etapa de 60 segundos, mantenha H atualizado durante a espera. Após reinício, o programa não conhece reservas físicas anteriores: a inspeção inicial precisa reconciliar toda a situação.

12. Registro que ajuda a investigar

Registre causa, horário, etapa, composição, rota, recursos reservados e última informação válida. Preserve o histórico antes de apagar o alerta. No exemplo, o Monitor Serial mostra somente mensagens e não mantém um arquivo persistente após reinício.

Defina também a reação à falha do próprio controlador ou da alimentação. Um aviso sonoro depende de energia, e uma mensagem só é útil se a interface continuar acessível. Verifique os mecanismos físicos e os procedimentos previstos na montagem.

13. Exercício e registro no Diário de Bordo

Escolha uma operação da ferrovia e liste três falhas possíveis. Para cada uma, descreva detecção, ação de parada, recursos que permanecem protegidos e inspeção necessária para rearmar.

Teste as ocorrências no Monitor Serial. Registre resultado esperado, observado e procedimento de recuperação. A meta é tornar o rearme uma decisão conferida, sem apagar a ocupação nem reiniciar a circulação automaticamente.